Queue-Aware versus Resource-Based Autoscaling on Amazon EKS: An Experimental Comparison of KEDA and HPA under Poisson and Bursty Workloads

Lucas Melo Rocha
NUSP 13680400
PSI5120 – Topics in Cloud Computing
Polytechnic School, University of São Paulo

***Abstract*—Message-driven applications can scale from resource pressure observed inside containers or from demand waiting outside them. This paper compares these two approaches on Amazon Elastic Kubernetes Service. The resource-based treatment uses the Kubernetes Horizontal Pod Autoscaler with CPU utilization, while the queue-aware treatment uses Kubernetes Event-Driven Autoscaling with the backlog of an Amazon Simple Queue Service queue. Both treatments execute the same worker image and are constrained to one through four replicas on the same cluster. Reproducible traces represent homogeneous Poisson arrivals and a two-state Markov-modulated Poisson process with the same target mean but different burstiness. The paired experiment evaluates end-to-end latency p95 and ready pod-seconds as primary outcomes, with backlog, queue wait, scale-out delay, and drain time as secondary outcomes. Inference uses one aggregate per run and paired bootstrap intervals instead of treating messages as independent replicates. Across 17 complete pairs, KEDA used 47.8% fewer ready pod-seconds under Poisson traffic and 29.9% fewer under MMPP traffic. This saving came with geometric-mean p95 latency ratios of 2.03 and 2.66 relative to HPA. Thus, the tested queue-aware configuration favored lower active capacity rather than lower tail latency. The complete lifecycle is automated with least-privilege identities, cost guards, raw-data retention, and cleanup auditing.**

***Index Terms*—Amazon EKS, autoscaling, HPA, KEDA, Kubernetes, MMPP, SQS.**

I. Introduction

Elasticity is a central promise of cloud computing, but a controller can react only to demand that its metric exposes. A CPU-based controller observes work after a Pod receives it. A queue-based controller can observe waiting work before a consumer starts processing it. This distinction may be immaterial under smooth traffic, yet important when short overload periods build a backlog faster than resource metrics and control loops react.

Kubernetes provides the Horizontal Pod Autoscaler (HPA), which adjusts a workload's replica count from resource, custom, or external metrics [1]. Kubernetes Event-Driven Autoscaling (KEDA) integrates external event sources with the Kubernetes scaling API. Its Amazon SQS scaler uses visible and, by default, in-flight messages to derive desired capacity [2]. Both ultimately manipulate the same Deployment, but their signals represent different points in the request path.

Prior work has examined HPA behavior, alternative resource metrics, traffic prediction, and adaptive policies [3], [4], [5], [6], [7]. Event-queue autoscaling has also been studied as a tail-latency and resource-efficiency problem [8]. However, the literature reviewed for this study did not provide a controlled HPA/CPU versus KEDA/SQS comparison under both memoryless and correlated burst arrivals.

This paper asks: how do CPU-based HPA and SQS-backlog-based KEDA differ in latency, backlog control, scaling responsiveness, and active Pod consumption when offered loads have similar means but different burstiness? The study makes four contributions:

- a paired comparison that changes the autoscaling policy while holding the application, cluster, queues, traces, and replica limits constant;
- a reproducible Poisson and two-state MMPP workload generator that stores every arrival offset and CPU service demand before execution;
- an analysis based on run-level aggregates, paired ratios, and bootstrap confidence intervals;
- a guarded and auditable EKS lifecycle that includes least-privilege IAM roles, cost controls, and resource deletion checks.

II. Background and Related Work

A. Resource-Based Horizontal Scaling

For CPU utilization, HPA periodically reads metrics from the Kubernetes Metrics API. In simplified form, its replica recommendation is

$$c_{t+1} = \left\lceil c_t \frac{u_{\text{observed}}}{u_{\text{target}}} \right\rceil, \tag{1}$$

where c_t is the current replica count and utilization is measured relative to each container's CPU request [1]. Tolerance, unavailable metrics, stabilization, and rate policies modify the practical behavior. HPA therefore observes a consequence of demand rather than the queue itself.

Nguyen et al. analyze elastic container orchestration with HPA and show the importance of resource configuration and workload conditions [3]. Casalicchio and Perciballi compare relative and absolute container metrics [7]. Balla et al. and Toka et al. propose adaptive approaches beyond a fixed reactive threshold [6], [5]. Phuc et al. add traffic awareness in an edge-computing setting [4]. These studies motivate careful metric selection and reproducible load control.

B. Queue-Aware Event-Driven Scaling

KEDA exposes an external metric and creates an HPA for the target workload. For the SQS scaler used here, the observed demand is

$$q_t = q_{\mathrm{visible},t} + q_{\mathrm{inflight},t}. \tag{2}$$

With a target of b messages per Pod, the central recommendation is approximately $\lceil q_t/b \rceil$, subject to polling, HPA behavior, and the configured replica bounds [2]. SQS counters are approximate, and AWS advises interpreting their semantics rather than treating them as exact transactional values [9].

Queue-aware control can react while work is waiting, but it requires access to the queue metric and a meaningful backlog target. Ezzeddine et al. frame event queue autoscaling around tail latency and resource-efficient placement [8]. The present work narrows the comparison to two widely deployable Kubernetes policies and one shared SQS consumer.

C. Bursty Arrival Processes

A homogeneous Poisson process has independent exponential inter-arrival times and a constant rate λ. It is useful as a memoryless baseline, but many real workloads exhibit correlated bursts. A two-state MMPP combines a Markov chain with state-dependent Poisson rates λ_L and λ_H [10]. If transition rates are r_{LH} and r_{HL}, the stationary high-state probability is

$$\pi_H = \frac{r_{LH}}{r_{LH} + r_{HL}}, \tag{3}$$

and the stationary mean arrival rate is

$$\bar{\lambda} = (1 - \pi_H)\lambda_L + \pi_H \lambda_H. \tag{4}$$

The experiment sets the Poisson rate and MMPP stationary mean to the same value, while preserving intervals in which MMPP demand exceeds one-Pod capacity.

III. Research Method

A. Questions and Outcomes

The experiment addresses three research questions:

- RQ1: Which policy provides lower per-run end-to-end latency p95 under Poisson and MMPP arrivals?
- RQ2: Which policy uses fewer ready pod-seconds over arrival and drain?
- RQ3: How do the policies differ in backlog, queue wait, scale-out delay, peak replicas, and drain time?

Table I
Frozen Experimental Configuration

Component	Configuration
Region	`us-east-1`
EKS nodes	one `c7i-flex.large`
Node scaling	disabled, one node fixed
Worker replicas	minimum 1, maximum 4
Worker CPU	request 200m, limit 400m
Worker memory	request 96 MiB, limit 192 MiB
HPA signal	CPU, target 60% of request
KEDA signal	visible plus in-flight SQS messages
KEDA target	5 messages per Pod
Polling interval	15 s
Scale-down stabilization	60 s
Input visibility timeout	120 s
Trace duration	180 s

End-to-end latency starts at the SQS service timestamp for the accepted input message and ends when the worker completes processing. Ready pod-seconds are the time integral of ready replicas from the start of arrivals until the last worker completion. These are primary outcomes. Secondary outcomes include queue wait p50, p95, and p99; maximum and integrated backlog; time to the first additional ready replica; drain time after the last accepted input; peak replicas; producer lag; duplicates; and missing results.

B. Experimental Design

The design is paired and factorial. Autoscaler has two levels, HPA and KEDA, and arrival process has two levels, Poisson and MMPP. Each pre-generated trace is replayed once under each policy. Treatment order is deterministically shuffled within each workload and seed. The experimental unit is a complete run, not a message.

Five paired seeds are evaluated first. Five additional seeds are required if an operationally invalid attempt occurs or if any 95% paired bootstrap interval for the two primary KEDA/HPA ratios extends beyond plus or minus 10% of its point estimate. The experiment stops after ten paired seeds regardless of that precision criterion. Invalid attempts remain available for diagnosis and are not included as valid observations.

C. Controlled Environment

Table I summarizes the frozen configuration. Both policies share one Deployment and differ only in the active autoscaler. KEDA and the worker use separate IAM roles for service accounts. This follows least privilege and avoids placing long-lived AWS credentials in Pods [11].

The cluster has public worker networking and no NAT Gateway. It creates no load balancer, database, dashboard stack, or node autoscaler. Four workers can consume at most 1.6 vCPU by cgroup limit, leaving nominal CPU capacity for Kubernetes, KEDA, and Metrics Server on the 2-vCPU node.

D. Workload Construction

The common mean arrival rate is 1.6 messages/s. CPU service demand is exponential with mean 0.2 CPU-s and is clipped to the interval from 0.005 to 1 CPU-s. At a 0.4-CPU

Pod limit, one continuously busy replica has an approximate capacity of 2 messages/s before overhead. Thus, the mean load is below one-replica capacity, while bursts can exceed it.

MMPP rates are 0.4 and 4 messages/s. Transition rates are 0.05/s from low to high and 0.10/s from high to low. Therefore $\pi_H = 1/3$ and the stationary mean is 1.6 messages/s. The chain starts in its stationary distribution. Because a 180-second realization can differ greatly from its stationary mean, deterministic rejection sampling accepts an MMPP trace only when its count is within 10% of the expected count. The base seed, accepted arrival seed, attempt, empirical rate, offsets, and demands are retained. This rule uses only offered-load counts and no autoscaling outcome.

E. Run Procedure

Before each run, the runner verifies that input and result queues are empty for three consecutive observations, exactly one worker is ready, the expected autoscaler is the only active policy, and its status is healthy. The system is idle for at least 30 seconds before the first arrival. The producer follows absolute monotonic deadlines and dispatches sends through a bounded thread pool, preventing one SQS network round trip from delaying later arrivals. It records scheduling lag when each sender starts its API call.

A collector long-polls the result queue while a monitor samples the Deployment, Pods, nodes, and both SQS queues every five seconds. Collection continues through a 15-second quiet window after all unique results arrive, which captures late duplicates. Configuration snapshots taken before and after the run detect changes to the Deployment or autoscaler specification.

A run is invalid if producer lag p95 exceeds 250 ms, monitoring has a gap longer than 15 seconds, a node becomes NotReady, a worker restarts, a Pod is persistently unschedulable, configuration changes, queue residue remains, clock sanity checks fail, or any expected unique result is missing. One automatic retry uses the same trace after both queues are purged and the SQS purge interval has elapsed.

F. Statistical Analysis

For metric Y, each seed and workload yields the paired ratio

$$R_i = \frac{Y_{i,\mathrm{KEDA}}}{Y_{i,\mathrm{HPA}}}. \tag{5}$$

The analysis reports all paired points, the median ratio, and the geometric mean ratio. A nonparametric paired bootstrap resamples complete ratios with replacement and uses the 2.5th and 97.5th percentiles for a 95% interval [12]. It also separates runs by treatment order as a sensitivity analysis. This avoids pseudoreplication from hundreds of correlated messages within one controller run.

Queueing relations provide consistency checks rather than a stationary model of the experiment. For c replicas with service rate μ, the reference utilization is $\rho = \lambda/(c\mu)$. Little's Law, $L = \lambda W$, is interpreted cautiously because MMPP arrivals, changing capacity, and finite traces make the system transient.

IV. Implementation and Reproducibility

The worker is a non-root Python container. It long-polls one SQS message, validates the protocol, consumes the requested CPU time, sends a structured result, and then deletes the input. The result contains SQS acceptance time, worker receipt and completion times, Pod and node names, receive count, service demand, and a work digest. CPU time is measured with `time.process_time()`, so cgroup throttling increases wall time without reducing requested CPU work.

AWS resources are defined by CloudFormation and `eksctl`. CloudFormation creates two SQS queues, one immutable ECR repository, and separate managed policies for the worker and KEDA. The cluster uses one managed node group, public networking, and no NAT Gateway. The image tag includes a hash of the complete worker build context, and the resolved image URI is retained in each run summary.

Creation scripts require `ALLOW_AWS_CHARGES=YES`. Cleanup independently requires `ALLOW_AWS_DELETE=YES`, verifies the target account and region, deletes IRSA roles before the cluster and foundation stack, and audits EKS, EC2, EBS, NAT, Elastic IP, load balancer, CloudFormation, SQS, ECR, IAM, and log resources. The design keeps the expected session cost below USD 2 and the explicit ceiling at USD 20.

The repository includes unit tests for deterministic traces, workload bounds, message validation, temporal integration, manifest parsing, paired analysis, and the sequential stopping decision. The worker image, CloudFormation template, and `eksctl` configuration are also validated before cloud creation.

A. Retained Artifacts and Auditability

Each completed attempt directory contains producer events, accepted result messages, Kubernetes and SQS monitor samples, duplicate records, and a machine-readable summary. Immutable traces are retained separately by pattern and seed. Analysis discovers summaries recursively but accepts only records marked as main-campaign and valid. It then joins policies by workload and seed, so an interrupted or unpaired attempt cannot silently enter a paired estimate.

The analysis writes both tabular CSV outputs and publication figures. Its campaign-status record distinguishes planned runs, valid completed runs, complete pairs, and cells not executed. Reproduction therefore does not depend on console logs or values transcribed from this paper. The final lifecycle audit uses service-specific list operations instead of assuming that successful stack deletion implies complete cleanup.

V. Results

A. Campaign Integrity

The measured campaign produced 35 valid main runs: 18 Poisson runs and 17 MMPP runs. These runs processed between 271 and 327 Poisson messages and between 260 and 310 MMPP messages. All expected messages produced exactly one accepted result. No run had a missing result, worker restart, NotReady node, persistent unschedulable Pod, or configuration change. Median producer lag p95 was 1.99

Table II
RETAINED WORKLOAD TRACE CHARACTERISTICS

Pattern	Traces	Jobs range	Mean rate	Median IAT CV
Poisson	9	271–327	1.627	0.992
MMPP	9	260–310	1.609	1.941

Table III
PAIRED PRIMARY OUTCOMES, KEDA/HPA RATIO

Pattern	Outcome	n	Geo. mean	95% CI
Poisson	Latency p95	9	2.033	[1.436, 2.810]
Poisson	Pod-seconds	9	0.522	[0.486, 0.560]
MMPP	Latency p95	8	2.659	[1.558, 4.728]
MMPP	Pod-seconds	8	0.701	[0.581, 0.823]

ms for Poisson and 2.75 ms for MMPP, far below the 250-ms validity threshold. The largest monitoring gap was 8.98 s, also below its 15-s threshold.

The first five pairs failed the predefined interval-precision criterion, so the campaign entered its five-seed extension. Extended cloud runtime and repeated short-lived login interruptions led to termination after nine complete Poisson pairs and eight complete MMPP pairs. One additional HPA/MMPP run had no KEDA counterpart and was excluded from all paired estimates. No value was imputed. Consequently, the results retain more replication than the initial stage but do not claim completion of the planned ten-pair maximum. The AWS cleanup audit subsequently found no remaining project resource.

Table II characterizes the nine generated traces of each pattern. The mean empirical rates were close, but median inter-arrival coefficient of variation was approximately twice as large for MMPP. Most MMPP messages arrived in the high state even though that state occupies one third of stationary time, as expected because arrivals are sampled more frequently while its rate is high.

B. Primary Outcomes

Table III reports KEDA/HPA ratios. A ratio above one indicates a larger outcome under KEDA. For Poisson traces, the geometric-mean p95 latency ratio was 2.03 (95% bootstrap interval 1.44–2.81). For MMPP traces, it was 2.66 (1.56–4.73). HPA therefore had lower p95 latency under both arrival patterns in this configuration. The intervals exclude one, although their width shows substantial between-trace variability.

The corresponding ready pod-seconds ratios were 0.522 (0.486–0.560) for Poisson and 0.701 (0.581–0.823) for MMPP. KEDA thus used 47.8% and 29.9% fewer ready pod-seconds, respectively. The Poisson pod-seconds interval met the predefined plus-or-minus 10% precision criterion. The other three intervals did not. Figures 1 and 2 expose every paired point rather than only the aggregate estimates.

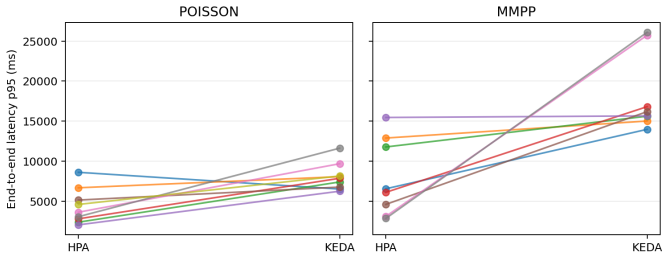

Figure 1. Per-trace p95 latency under HPA and KEDA. The dashed diagonal marks equal outcomes; points above it favor HPA.

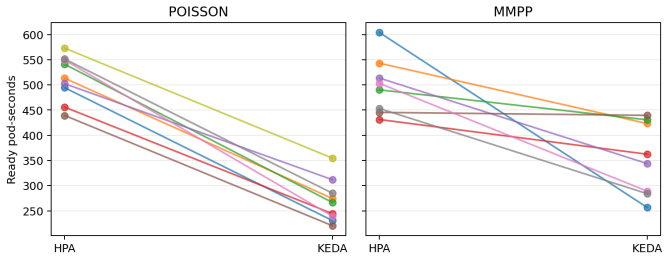

Figure 2. Per-trace ready pod-seconds. Points below the equal-outcome diagonal indicate lower active Pod consumption under KEDA.

C. Scaling and Backlog Behavior

The descriptive measurements explain the primary trade-off. Under Poisson traffic, median p95 latency was 3.61 s for HPA and 7.85 s for KEDA, while median ready pod-seconds were 513 and 267. Median peak replica count was four for HPA and two for KEDA. The median maximum backlog was nine messages under HPA and 15 under KEDA. Median time until an additional Pod became ready was 36.8 s for HPA and 81.5 s for KEDA.

Under MMPP traffic, median p95 latency across valid runs was 6.54 s for HPA and 15.91 s for KEDA. Median ready pod-seconds were 498 and 353. Both policies had a median peak of four replicas, but median maximum backlog was 19 for HPA and 39 for KEDA. The KEDA backlog target therefore did not translate into earlier ready capacity. Its 15-s polling cadence, external-metric path, and the finite time needed to start a Pod allowed more waiting work to accumulate before capacity arrived.

Figure 3 shows that MMPP outcomes were heterogeneous. KEDA controlled maximum backlog on some traces but accumulated much larger peaks on others. This variation is consistent with sensitivity to the timing and length of high-state intervals, which equal message counts alone do not control.

Treatment-order sensitivity was stable for resource use but not for latency. For Poisson, median pod-seconds ratios were 0.517 when HPA ran first and 0.518 when KEDA ran first. For MMPP they were 0.627 and 0.780. Latency ratios were larger when HPA ran first, especially for the final three MMPP pairs. Since order and late campaign time are partly confounded in this small sample, this pattern cautions against assigning all

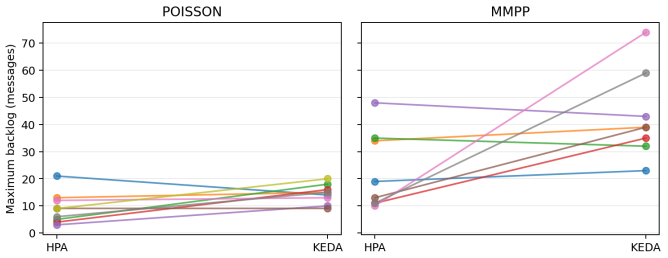

Figure 3. Per-trace maximum observed backlog. Lower points indicate stronger backlog containment during the finite workload.

observed latency variation solely to the controller.

VI. Discussion

The experiment does not support the intuitive claim that observing the queue necessarily improves tail latency. KEDA observed a signal closer to external demand, but the selected target of five messages per Pod preserved fewer active Pods during the mostly sub-capacity mean load. HPA's 60% CPU target instead kept or requested more capacity once work reached a consumer. In this setting, that aggressive capacity response outweighed the informational advantage of the queue signal.

The cost-latency trade-off was consistent in direction across both workloads. Every aggregate resource estimate favored KEDA, while aggregate p95 latency favored HPA. Burstiness reduced KEDA's relative resource saving from about 48% to 30% because MMPP high states forced both policies toward the four-replica ceiling. It did not reverse the latency result. Instead, several burst traces produced especially large KEDA/HPA latency ratios, accounting for the wide MMPP interval.

These findings apply to policy configurations, not controller brands in the abstract. A smaller KEDA backlog target, shorter polling interval, higher minimum replica count, or predictive activation could spend more pod-seconds to reduce waiting. Conversely, a higher HPA CPU target or longer stabilization could save capacity at a latency cost. The practical implication is that a queue-aware metric still requires a target aligned with a service-level objective; metric proximity alone does not guarantee responsiveness.

A simple capacity calculation clarifies the controller incentives. Mean service demand is 0.2 CPU-s and one Pod is limited to 0.4 CPU, giving an idealized service rate near 2 messages/s. At the common mean arrival rate of 1.6 messages/s, one Pod has reference utilization $\rho \approx 0.8$. This already exceeds HPA's 60% CPU target when the worker remains busy, encouraging HPA to add capacity. KEDA, in contrast, requests capacity only after the observed queue exceeds its per-Pod backlog target. During the MMPP high state, the arrival rate rises to 4 messages/s and at least two idealized Pods are needed merely to match arrivals. Any polling and startup delay then creates a transient queue that additional Pods must drain. This calculation is not a stationary performance model, but it matches the observed direction of scale-out and backlog differences.

The results also illustrate why per-message significance tests would be misleading. Hundreds of messages in one trace share the same burst history, controller state, node, and Pods. Pairing at trace level yields only eight or nine independent contrasts per workload and appropriately exposes uncertainty and temporal sensitivity.

VII. Threats to Validity

The experiment uses one AWS region, one cluster, one node type, one worker implementation, and one set of controller parameters. Results therefore do not establish universal superiority. A different service-time distribution, image startup time, backlog target, CPU request, or polling interval may change the comparison.

SQS queue attributes are approximate and sampled every five seconds. Timestamp subtraction relies on AWS service time and the EKS node clock; negative values beyond a small tolerance invalidate a run, but small positive clock offsets cannot be separated from queue wait. Pod-seconds use zero-order-hold integration between samples and are consequently discretized.

Immediate post-run queue counters are retained but are not used alone as an invalidity condition because SQS exposes eventual approximations. Instead, the next run requires three consecutive empty observations after a bounded wait, and the collector rejects results carrying another experiment identifier.

The fixed node removes node-autoscaling variation but caps total capacity. CPU limits reserve nominal capacity for system components, and persistent unschedulability or node readiness failures invalidate a run. Five or ten pairs support effect estimation but only limited inference. Paired bootstrap intervals and visible individual points reduce, rather than eliminate, this limitation.

The extension stopped with nine Poisson and eight MMPP pairs rather than ten of each. This early termination weakens precision and leaves treatment order partly confounded with campaign time, particularly for MMPP latency. Reporting the unpaired run only in descriptive validation, retaining all raw attempts, and making no imputation limit but do not remove this threat.

Conditioning MMPP traces on total count improves comparability of offered load but excludes unusually high- or low-count short realizations. The temporal clustering remains stochastic and recorded. Finally, treatment order is randomized deterministically and analyzed, but all runs still share one cluster, so unmeasured temporal drift may remain.

VIII. Conclusion

For the tested EKS worker and controller settings, CPU-based HPA provided lower end-to-end p95 latency, whereas SQS-based KEDA consumed fewer ready pod-seconds. KEDA/HPA geometric-mean latency ratios were 2.03 under Poisson and 2.66 under MMPP. Ready pod-seconds ratios were 0.522 and 0.701, equivalent to reductions of 47.8% and

29.9%. KEDA also had larger median backlogs and slower ready scale-out in this configuration, so queue visibility alone did not overcome polling, Pod startup, and target selection.

The result is a configuration-specific trade-off rather than a universal policy ranking. It demonstrates a reproducible way to compare internal resource pressure with external queued demand using frozen traces, run-level pairing, validity gates, retained raw observations, and audited cloud cleanup. Future work should vary the backlog and CPU targets, add longer traces and independent clusters, and complete a larger randomized campaign to separate treatment order from temporal drift.

REFERENCES

[1] Kubernetes Authors, "Horizontal pod autoscaling," accessed: 2026-08-31. [Online]. Available: https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/

[2] KEDA Authors, "AWS SQS Queue Scaler," accessed: 2026-08-31. [Online]. Available: https://keda.sh/docs/2.17/scalers/aws-sqs/

[3] T.-T. Nguyen, Y.-J. Yeom, T. Kim, D.-H. Park, and S. Kim, "Horizontal pod autoscaling in Kubernetes for elastic container orchestration," *Sensors*, vol. 20, no. 16, p. 4621, 2020.

[4] L. H. Phuc, L.-A. Phan, and T. Kim, "Traffic-aware horizontal pod autoscaler in Kubernetes-based edge computing infrastructure," *IEEE Access*, vol. 10, pp. 18 966–18 977, 2022.

[5] L. Toka, G. Dobreff, B. Fodor, and B. Sonkoly, "Adaptive AI-based auto-scaling for Kubernetes," in *2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID)*, 2020, pp. 599–608.

[6] D. Balla, C. Simon, and M. Maliosz, "Adaptive scaling of Kubernetes pods," in *NOMS 2020 – 2020 IEEE/IFIP Network Operations and Management Symposium*, 2020, pp. 1–5.

[7] E. Casalicchio and V. Perciballi, "Auto-scaling of containers: The impact of relative and absolute metrics," in *2017 IEEE 2nd International Workshops on Foundations and Applications of Self* Systems (FAS*W)*, 2017, pp. 207–214.

[8] M. Ezzeddine, F. Baude, and F. Huet, "Tail-latency aware and resource-efficient bin pack autoscaling for distributed event queues," in *Proceedings of the 14th International Conference on Cloud Computing and Services Science (CLOSER)*, 2024, pp. 50–64.

[9] Amazon Web Services, "Available CloudWatch metrics for Amazon SQS," accessed: 2026-08-31. [Online]. Available: https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-available-cloudwatch-metrics.html

[10] W. Fischer and K. Meier-Hellstern, "The markov-modulated poisson process (MMPP) cookbook," *Performance Evaluation*, vol. 18, no. 2, pp. 149–171, 1993.

[11] Amazon Web Services, "IAM roles for service accounts," accessed: 2026-08-31. [Online]. Available: https://docs.aws.amazon.com/eks/latest/userguide/iam-roles-for-service-accounts.html

[12] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993.